MERGE SORT & QUICK SORT – C PROGRAM

Input: 324, 125, 456, 218, 102, 389, 275, 147

PROGRAM	OUTPUT

```c
#include <stdio.h>

int mergeComparisons = 0, quickComparisons = 0;

void merge(int a[], int low, int mid, int high) {
    int i=low, j=mid+1, k=0, temp[20];
    while(i<=mid && j<=high) {
        mergeComparisons++;
        if(a[i] < a[j]) temp[k++] = a[i++];
        else temp[k++] = a[j++];
    }
    while(i<=mid) temp[k++] = a[i++];
    while(j<=high) temp[k++] = a[j++];
    for(i=low,k=0; i<=high; i++,k++) a[i]=temp[k];

    printf("After merge: ");
    for(i=low;i<=high;i++) printf("%d ",a[i]);
    printf("\\n");
}

void mergeSort(int a[], int low, int high) {
    if(low < high) {
        int mid=(low+high)/2;
        mergeSort(a,low,mid);
        mergeSort(a,mid+1,high);
        merge(a,low,mid,high);
    }
}

int partition(int a[], int low, int high) {
    int pivot=a[high], i=low-1, j, temp;
    for(j=low;j<high;j++) {
        quickComparisons++;
        if(a[j] < pivot) {
            i++;
            temp=a[i]; a[i]=a[j]; a[j]=temp;
        }
    }
    temp=a[i+1]; a[i+1]=a[high]; a[high]=temp;

    printf("Pivot %d: ",pivot);
    for(j=low;j<=high;j++) printf("%d ",a[j]);
    printf("\\n");
    return i+1;
}

void quickSort(int a[], int low, int high) {
    if(low < high) {
        int p=partition(a,low,high);
        quickSort(a,low,p-1);
        quickSort(a,p+1,high);
    }
}

int main() {
    int a[]={324,125,456,218,102,389,275,147};
    int b[]={324,125,456,218,102,389,275,147};
    int n=8, i;

    printf("MERGE SORT\\n");
    mergeSort(a,0,n-1);
    printf("Final: ");
    for(i=0;i<n;i++) printf("%d ",a[i]);
    printf("\\nComparisons = %d\\n",mergeComparisons);

    printf("\\nQUICK SORT\\n");
    quickSort(b,0,n-1);
    printf("Final: ");
    for(i=0;i<n;i++) printf("%d ",b[i]);
    printf("\\nComparisons = %d\\n",quickComparisons);

    return 0;
}
```

```
MERGE SORT
After merge: 125 324
After merge: 218 456
After merge: 102 389
After merge: 147 275
After merge: 125 218 324 456
After merge: 102 147 275 389
After merge: 102 125 147 218 275 324 389 456
Final: 102 125 147 218 275 324 389 456

QUICK SORT
Pivot 147: 125 102 147 218 456 389 275 324
Pivot 102: 102 125
Pivot 324: 218 275 324 456 389
Pivot 275: 218 275
Pivot 389: 389 456
Final: 102 125 147 218 275 324 389 456
```

ANALYSIS

Merge Sort: O(n log n) time, O(n) extra space.
Quick Sort: Average O(n log n), worst O(n^2), O(log n) average extra space. For large fixed-length keys, Merge Sort gives predictable O(n log n) performance.

Final sorted sequence: 102, 125, 147, 218, 275, 324, 389, 456